



Data structure and Programming

TP08: Stack data structure

Deadline:
6 days

- Objective:
 - To practice create Stack data structure that is able to store unlimited data and data is stored via LIFO method
 - Implement push and pop methods for stack data structure.
 - Apply C++ programming
- Individual work
- Submit to Moodle (do not zip)
 - A pdf report (cover, exercise and solution with screenshot)
 - Source codes

1. Write a C++ program to implement a stack data structure using an array to store integer numbers. Write functions for: push(), pop(), peek(), isEmpty(), isFull(), displayStack(). Test the implementation by pushing and popping several elements.
2. Write a C++ program to implement a stack data structure using a singly linked list to store integer numbers with the following operations: push(), pop(), peek(), isEmpty(), and function to count and return the number of elements in the stack, and display function to print all elements in the stack from top to bottom.
3. Write a program to ask a piece of text from a user. Display the text in a reverse order using stack.
Hint: Push each character in the text into stack. Display stack.
4. Write a program to get a number from a user. Use stack to store that number as the binary number representation in the stack. Finally, display the binary representation from the stack.
Hint: Divide the integer by 2 then store the remainder in stack. Finally, display data in stack.
5. A letter means push and an asterisk means pop in the following sequence. Give the sequence of values returned by the pop operations (display top data in queue then delete it) when this sequence of operations is performed on an initially empty LIFO stack.

E A S * Y * Q U E * * * S T * * * I O * N * * *

6. Write a program that reads a piece of delimiter symbols containing parentheses, braces and/or curly braces. The program determines whether its parentheses, braces, and curly braces are "balanced". The program can be done using stack data structure.

Hint: Process each character of the input.

- When a character is left delimiter, push it to stack.
- When a character is right delimiter, pop data from stack and check whether popped element matches right delimiter.
- It is balance when all matching are true and stack is empty when all characters have been processed.

Examples of balanced and unbalanced pieces of symbols:

(({})) **Balanced!**

{ () } **Not balanced!**

((() }) **Not balanced**

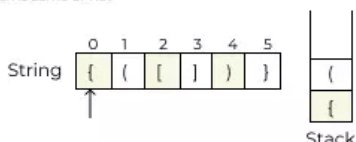
[()] **Balanced**

Check for balanced parentheses		
Expression	Balanced?	() or {} or []
) (	NO	
[()]	Yes	
[(])	NO	
[() ()]		
⇒ Last unclosed, first closed		

Balanced Parenthesis Problem

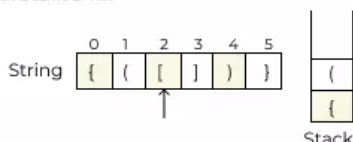
Step 1

Closing brackets Check top of the stack is same or not



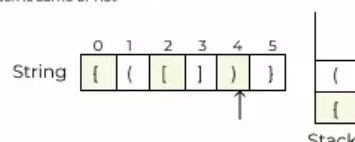
Step 3

Closing brackets Check top of the stack is same or not



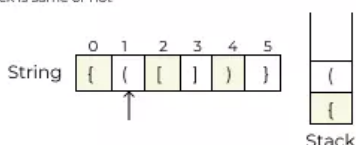
Step 5

Closing brackets Check top of the stack is same or not



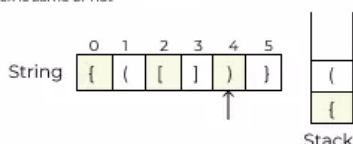
Step 2

Closing brackets Check top of the stack is same or not



Step 4

Closing brackets Check top of the stack is same or not



Step 6

Closing brackets Check top of the stack is same or not

